MP4-G82 REPORT

bganesh2 : Ganesh, Bharath
yonghan4 : Chen, Yong-Han

a. Design

RainStorm adopts a Leader-Worker architecture:

Leader (VM1): Handles scheduling and fault tolerance. It uses Round-Robin to distribute tasks to workers and creates timestamped Dedup Logs in HyDFS. It triggers all tasks followed by the source. The leader maintains a GlobalTaskMap for scaling and broadcasting upstream and downstream by tracking task info. It also uses a TaskRates map to monitor loads for autoscaling. The leader handles the End-of-Stream (EOS) signal and restarts the task when the task was killed.

Worker: Executes operators and manages local task state. Workers parse operator output (key-value pairs via \t) and route tuples to the next stage. We implement Hash Partitioning with 32-bit FNV-1a for tuple routing. This ensures that tuples with the same key are consistently sent to the same downstream task, ensuring correctness for stateful operators like AggregateByKey. Tasks in the final stage append results to the HyDFS destination file.

Exactly-Once Semantics: We achieve exactly-once by ensuring at-least-once and at-most-once

At-least-once

RPC Retries: Upstream tasks (including Source) use a blocking retry mechanism. If a downstream task fails, the sender holds the tuple and retries PushTuple until the leader restarts a new healthy task.

Log Replay: When receiving a tuple, tasks write MessageID and content to a Dedup Log in HyDFS before processing. If the operator was killed, the restarted task first reads and replays this log to stdin to ensure no data loss between stdin and stdout during the operator's execution.

At-most-once

To filter duplicate tuples caused by retries or replays, each task maintains a map using the unique MessageID (StageID-TaskID-SeqNum) as the key. Any incoming tuple with an existing key in the map is discarded. Moreover, during recovery, a restarted task first reconstructs this deduplication map by reading the Dedup Log before replaying data to stdin, ensuring that retried tuples are not processed as new inputs.

Autoscaling

Monitoring and Decision: Each task periodically reports its processing rate to the Leader. The Leader filters out stale reports (zombies) and updates rates. The Leader monitors average rates against High (HW) and Low (LW) Watermarks. To prevent oscillation, scaling actions are at-most-once per round and enforced by cooldown periods (1s for Scale Up, 3s for Scale Down).

Scale Up: The Leader selects the alive worker with the least load to host the new task, creates a new Dedup Log, and starts a new task. The Leader broadcasts updated downstream addresses to the previous stage (for data routing) and updated upstream counts to the next stage.

Scale Down: The Leader identifies the victim (Max TaskID) and updates metadata before killing the task to prevent data loss. It performs a blocking broadcast to update the previous stage's address to prevent upstream tasks from sending new tuples (at-least-once) and update the next stage's upstream count. Since we don’t kill tasks when there is only one task, we assume Task 0 is always alive. To ensure at-least-once semantics, Task 0 replays the Dedup Log of the killed task to prevent data loss.

End of Stream (EOS): The Source stage emits EOS tuples when completing the input file. Each downstream task tracks the number of active upstream tasks. A task waits to receive EOS tuples from all its upstream tasks before sending an EOS tuple to the next stage. Then, each task sends a

finish message to the Leader. The Leader marks the job as completed only when the number of finished tasks equals TotalTasks. EOS supports autoscaling by dynamically updating the upstream counts for downstream tasks and TotalTasks during scaling.

b. Past MP Use

We used MP3 to write, append, and get Dedup Logs and output files, while MP2 maintained the alive node list required for task scheduling and autoscaling. In addition, MP1 is essential for debugging autoscaling and recording rainStorm completion time.

c. LLM Use

We used LLM to achieve exactly-once by integrating RPC retries and Log Replay. In addition, it helped resolve a race condition during Scale-Up by suggesting timestamped Dedup Logs. Moreover, the LLM assisted in verifying Autoscaling correctness by analyzing data patterns (e.g., "SIGN_GUIDE" in dataset2.csv) to predict expected Scale-Up/Down sequences.

d. Measurements

Fig1: Average Throughput Comparison across 2 Applications: Spark vs RainStorm
Dataset: Instagram_Analytics.csv

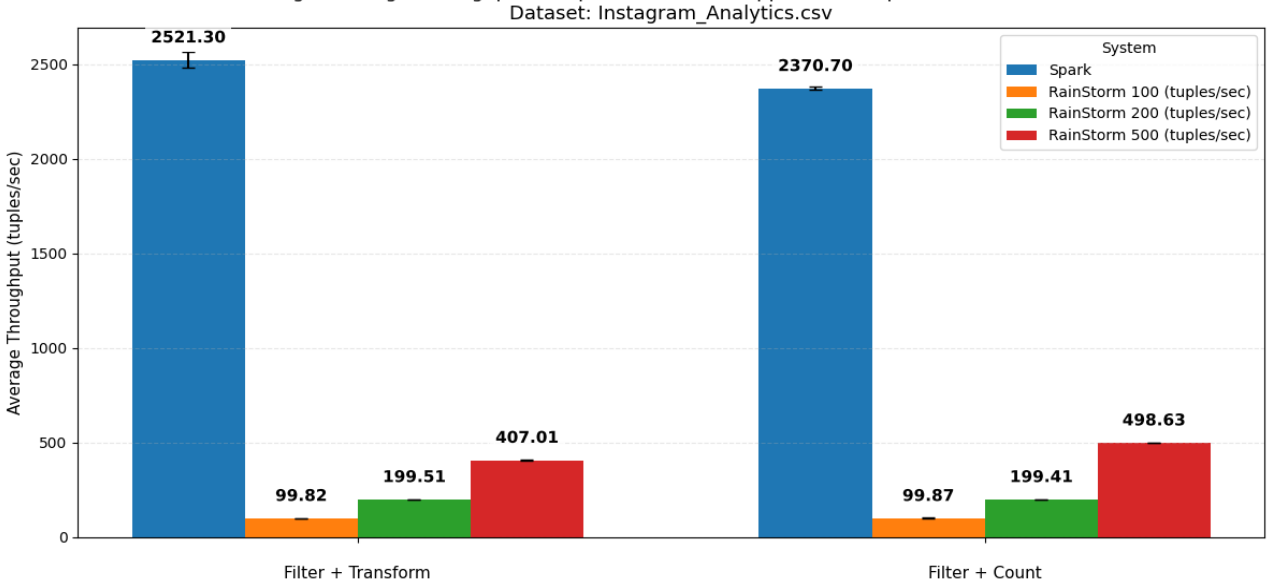
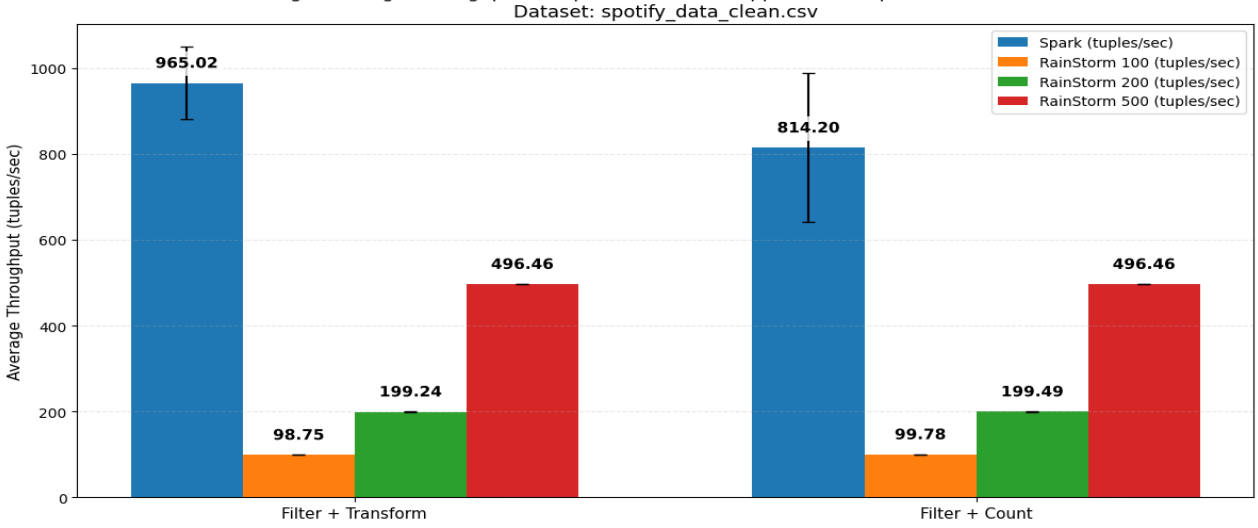


Fig2: Average Throughput Comparison across 2 Applications: Spark vs RainStorm
Dataset: spotify_data_clean.csv



For our experiments we used two public Kaggle datasets. The Instagram Analytics dataset contains 29,999 rows of post-level metrics such as likes, comments, shares, reach, impressions, engagement rate and media type [1]. The Spotify Global Music dataset contains 8,582 rows of track and artist information, including track name, artist, album, popularity and audio features from the Spotify API [2]. In every experiment we kept our RainStorm configuration fixed at two stages and three tasks per stage (N_Stage = 2, N_Task = 3) and ran the system in EXACTLY_ONCE mode. To study how RainStorm behaves under different loads, we varied the input rate to 100, 200 and 500 tuples per second and measured how throughput changed. We have defined throughput as total input lines processed by the total execution time.

We implemented 2 streaming applications, two for each dataset. For Instagram, the first application used Filter + Count, selecting only posts with media type “Reels” and calculating count group by column 14. The second used Filter + Transform, selecting posts containing the word “Photography” and then applying a transform that kept only the first three columns of each row. For the Spotify dataset, one application used Filter + Count, filtering rows containing “hop” and calculating count group by column 13. The second used Filter + Transform, allowing all rows to pass filter patter “TRUE” and applying the same transformation that outputs only the first three columns.

Across both datasets, the throughput results shown in Fig. 1 (Instagram) and Fig. 2 (Spotify) follow the same overall pattern.

Method Comparison
Spark achieves higher throughput because it reads data directly from local storage and processes it in micro-batches, while RainStorm reads from HyDFS and pays a network delay for every block of input. RainStorm shows minimal variation because it was controlled by a fixed input rate in the Source stage.

Dataset Comparison
Spark throughput increases with the Instagram dataset, proving that batch optimization works for larger datasets. However, for input rate 500, filter + transform application, RainStorm throughput (407 tuples/sec) for Instagram dataset is lower than the Spotify dataset (496 tuples/sec). The hash function might not have uniformly distributed tuples to the task, which might have congested tuples from the filter stage for the Instagram dataset preventing throughput from aligning with the input rate (500).

Application Comparison
For Spark, Filter + Count (Stateful) have smaller throughput in comparison to Filter + Transform (Stateless), for bath datasets. This shows that state management and data shuffling during aggregation are too expensive to have the same throughput as stateless operation. RainStorm's performance remained consistent across applications, since the input rate was not high enough to expose the state management overhead as a primary bottleneck.

[1] https://www.kaggle.com/datasets/kundanbedmutha/instagram-analytics-dataset
[2] https://www.kaggle.com/datasets/wardabilal/spotify-global-music-dataset-20092025